

Prompt Engineering

Dr. Gaurav Dixit

Why Prompt Engineering?

Prompt Engineering

- Practice of constructing and optimizing a prompt (text input in natural language) to interact with an LLM (large language model) to improve its performance on a task at hand
 - A prompt can steer the model towards *generating* a desired output
- In many cases, even model fine-tuning for a task is not needed

It all comes down to writing a good prompt

Large Language Model (LLM)

Large Language Models (LLMs)

- Generative AI models that predict the next word in a block of text
- Most powerful tool of the modern era

Generative AI (GenAI)

- Artificial Intelligence that is capable of generating text, images, and other types of content

Large Language Model (LLM)

Generative AI

- Power to democratize AI makes it a great technology
- Anyone can use it by typing or voicing a sentence in natural language
- Examples:
 - **Complex task:** search for answers via queries on the web search
 - **Simplified:** Ask an LLM to talk about a topic

Large Language Model (LLM)

Generative AI

- Examples:

- **Complex task:** find answers to technical queries on **Stack Overflow**
- **Simplified:** Ask an LLM to give a stepwise solution and follow it up with more Q&A

Large Language Model (LLM)

Generative AI

- Examples:

- **Complex task:** manually develop code for a software program from scratch
- **Simplified:** pair with a GenAI assistant to auto-generate the boilerplate code and handle the rest with completion

Large Language Model (LLM)

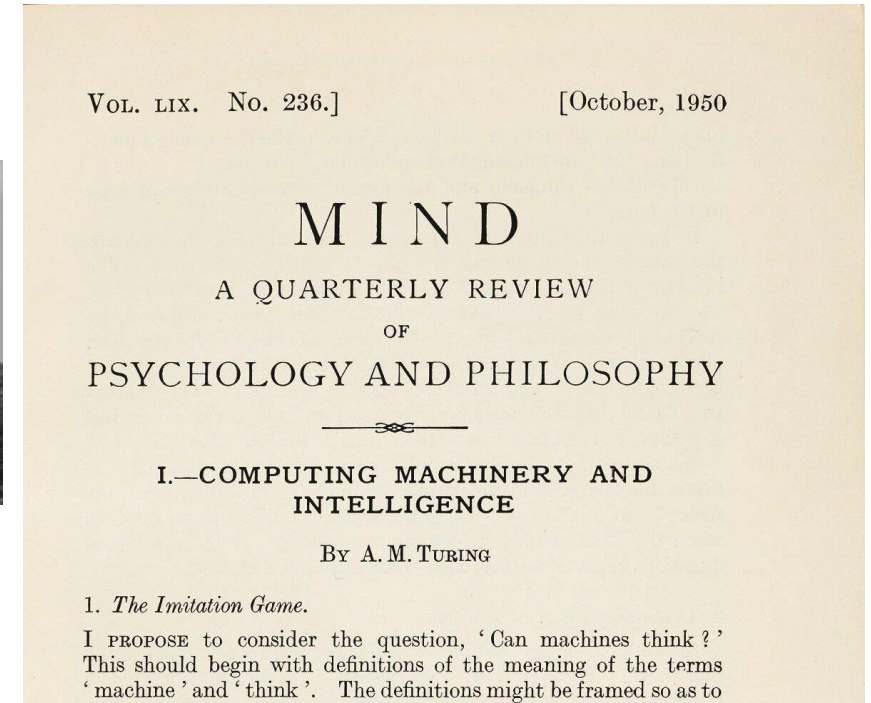
How did we reach this great milestone in the history of modern technology?

- History of AI

History of AI: 1950

Starting point:

- Alan Turing's landmark paper (1950)
 - Computing Machines and Intelligence
- Turing asked the question:
"Can machines think?"



Turing Test

- Imitation Game, known as the Turing Test

- A machine is said to pass the Turing test

If it can *convince* a human judge that

It's actually a human through *natural language dialogue*

History of AI

1956: The notion of Artificial Intelligence was conceived

1990s: Machine Learning (Statistical AI)

- Support Vector Machines

2010s: Deep Learning (Neural AI)

- Neural Networks

2020s: Large Language Model (Generative AI)

- Transformers

Artificial Intelligence

Machine Learning

Deep Learning

Generative AI

1956

Artificial Intelligence

the field of computer science that seeks to create intelligent machines that can replicate or exceed human intelligence

1990s

Machine Learning

subset of AI that enables machines to learn from existing data and improve upon that data to make decisions or predictions

2010s

Deep Learning

a machine learning technique in which layers of neural networks are used to process data and make decisions

2020s

Generative AI

Create new written, visual, and auditory content given prompts or existing data.

History of AI

1986: popularized **backpropagation** to train multi-layer neural networks

- By Donald Rumelhardt, Geoff Hinton, and Ronald Williams

1989: system based on **convolutional neural networks** to recognize handwritten digits

- By Yann LeCun

History of AI

2006: unsupervised layerwise pre-training of deep networks

- By Geoff Hinton and colleagues

2012: Trained a landmark convolutional neural network called AlexNet

- By Alex Krizhevsky, Ilya Sutskever, and Geoff Hinton

History of AI

2017: Transformer architecture

- By Ashish Vaswani and colleagues

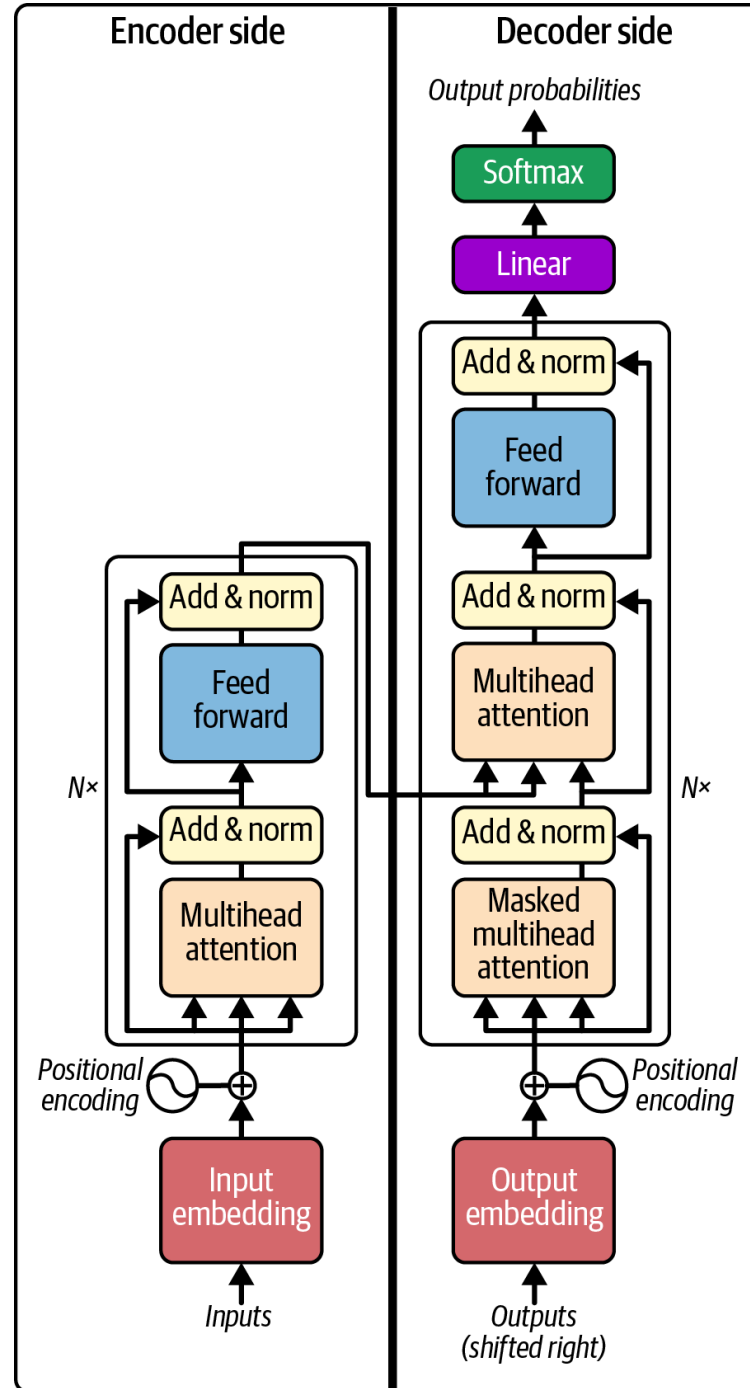
2022: InstructGPT (ChatGPT: sibling model)

- By Long Ouyang and colleagues

History of AI

InstructGPT

- Pioneered **RLHF** (Reinforcement Learning from Human Feedback)
- **Training technique** that turned a raw, unpredictable AI into an easy-to-use **conversational chatbot**



Large Language Model (LLM)

How do LLMs work?

- Receive an input text and generate an output text
- **Statistical models**: work much better with **numbers** than text sequences
- **Tokenization**: data processing via **Tokenizer**
 - Every **input** to the model is **processed** by a tokenizer before being used by the core model

Large Language Model (LLM)

Token

- A **chunk of text** consisting of a variable number of characters

Tokenizer's Main Task: text → *numbers*

- First, **split/chunk** the input (text prompts) into an **array of tokens**
- Then, each token is **mapped** to a **token index**
 - **Integer encoding** of the original text chunk



VISUAL BREAKDOWN: TOKENIZATION OF 'LEARNING AI IS FUN!'



[Learning AI is fun!]

📄 Characters: 19 | Tokens (LLaMA): 4 | 💬 Tokens (GPT-4): 5



LLaMA Token Blocks (4 Tokens)



LLaMA: Merges 'fun' and '!'.



GPT-4 Token Blocks (5 Tokens)



GPT-4: Separates 'fun' and '!'.



Large Language Model (LLM)

Predicting Output Tokens:

- Given n tokens as input →

Model predicts 1 token as output

- Models have a max token length that varies across models
- Model pricing is also based on the usage of #tokens

Large Language Model (LLM)

Expanding Window Pattern:

- Predicted token is added to the input of the next iteration
- Enables a better user experience:
 - Users can get a one-sentence or a multi-sentence answer

Large Language Model (LLM)

Selection Process via Probability Distribution:

- Based on its training, the model predicts

A probability distribution over all possible next tokens

- Deterministic Behavior:

- The model chooses the output token based on its probability of occurring after the current text sequence

Large Language Model (LLM)

Selection Process via Probability Distribution:

- Non-deterministic Behavior:

- Do not get the exact same output for the same input

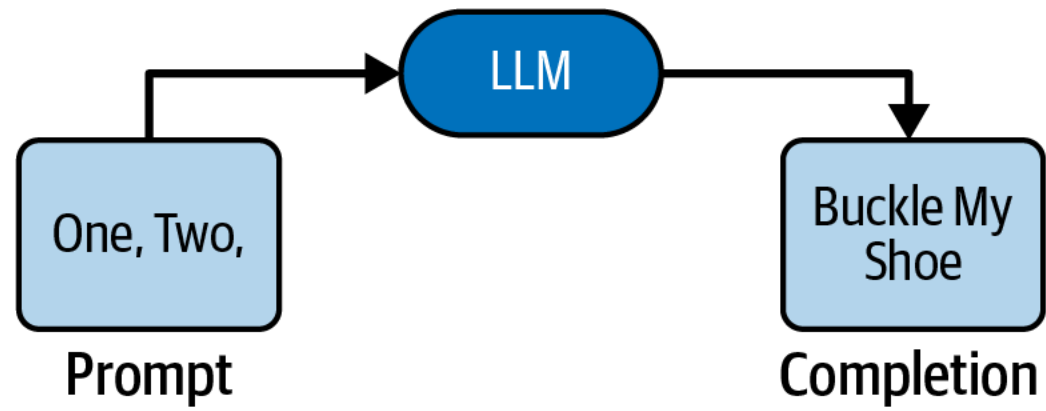
- Simulate the creative thinking process

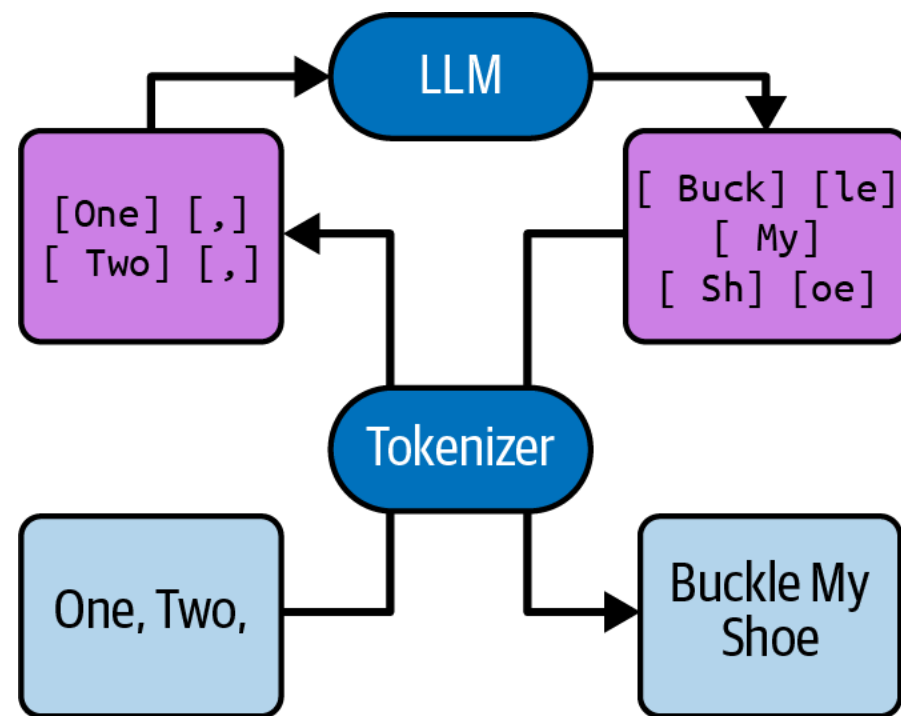
- Model choice is influenced by adding a degree of randomness
- Tunable via a model hyperparameter called temperature

Prompt Engineering

LLM Mechanism:

- Generates the next token to **complete** the current input
- Prompt: Input of an LLM
 - Typically, text input in **natural language**, sometimes voice input
- Completion: Output of an LLM
 - Model's prediction of next token





Prompt Engineering

Prompt Construction Forms:

- **Instruction** for expected output
 - Embed (Optional): example, data, or code
- Conversational **Question**
- Text that needs completion
- Comment asking for code generation for a task

Hands-on

Task: Summarization

e.g., article, book, product reviews

- Extraction of insights from unstructured data

Open <https://chatgpt.com/>

Prompt: Summarize quantum computing

Hands-on

Task: Conversational Question

Prompt: What are the most used alternatives to ChatGPT for consumers? Give URLs

Hands-on

Task: Creative ideation and design
e.g., article, essay, assignment

Prompt: Write an assignment for high school students, including King Vikramaditya of Ujjain and his judgments

Hands-on

Task: Code **generation** for a task

Prompt: A Python function to find a prime number

LLM Providers

LLM ecosystem

<https://huggingface.co/chat/>

Multi-LLM experimentation

<https://openrouter.ai/chat>

Free API playground

https://aistudio.google.com/prompts/new_chat

Large Language Model (LLM)

Types of LLMs:

- Categorizations can be based on
 - Data type, Output, Architecture
- Choice of LLM model
 - Application, data modality, willingness to pay

Foundation Models

Foundation Models

- **Large-scale** AI models trained on *massive*, *diverse* datasets to perform a wide variety of downstream tasks
 - Serve as a '*foundation*' for other models
 - Training via unsupervised learning or self-supervised learning
 - Very large models, billions of parameters
- e.g., in 2022, GPT-3.5 was the foundational model for ChatGPT

Foundation Models

Foundation Models

- Language Models (Text): Chat, translation, reasoning
- Vision Models (Images/video): Detection, recognition
- Audio Models (Speech/audio): Transcription, synthesis
- Multimodal Models (Mixed data): Cross-modal understanding
- Generative Models (Any modality): Content creation

Popular Models

Audio and speech recognition

- OpenAI's Whisper V3/Turbo, NVIDIA Canary-Qwen 2.5B

Image generation

- OpenAI's GPT Image 2, Midjourney V8/V8.1

Popular Models

Text generation

- OpenAI GPT-5.5 / 5.2 / o3, Anthropic Claude 4.5, Google Gemini 3.1 Pro / 3 Pro

Multi-modality

- Google Gemini 3.1 Pro, OpenAI GPT-5.5, Anthropic Claude 4.5

Large Language Model (LLM)

Open Source LLMs

- Available for public use by anyone
- Can be inspected, modified, and customized
- Not always optimized for production
- May not be as performant as proprietary models
- Limited funding: long-term maintenance or latest updates difficult
- **Examples:** Alpaca, Bloom, LLaMA

Large Language Model (LLM)

Proprietary LLMs

- Not available for public use
- Not allowed to be inspected, modified, or customized
- Often optimized for production
- Not always available for free: subscription or payment to use
- No control over the training data: trust the model owner
- **Examples:** OpenAI models, Google Bard, Claude 2

Large Language Model (LLM)

Embedding Models (Embedders)

- Convert text into a numerical form, called **embedding**
 - Capture the '*semantic meaning*' or '*context*' of the input data
- **Embeddings**: **numerical representations** of the input text
 - Easier for machines to understand the relationships between words or sentences
 - Can be consumed as inputs by models for better performance

Large Language Model (LLM)

Embedding Models (Embedders)

- Often used for **transfer learning**
 - A model is built for a *surrogate task* for which there's an abundance of data
 - Model weights (embeddings) are reused for *downstream tasks*
- e.g., OpenAI embeddings

**"Apple is a
tech company"**

string

Converter (BERT)

Dense Vector
(e.g., 768 dimensions)

0.137	-0.421 -0.442	0.805 -0.031	-0.092 -0.421	...	0.211	-0.503
-------	------------------	-----------------	------------------	-----	-------	--------

Embedding

Large Language Model (LLM)

LLM categorization by output

- Image Generation Models

- Used for image editing, image synthesis, and image translation
- Often trained on large image datasets, such as LAION-5B
- Generate new images or edit existing images with inpainting, super-resolution, and colorization techniques

e.g., DALL-E-3, Stable Diffusion models

Hands-on

Task: Image generation

Prompt: An image of a comet

Large Language Model (LLM)

LLM categorization by output

- Text Generation Models

- Used for text summarization, translation, and question answering
- Often trained on large text datasets, such as BookCorpus
- Generate new text or answer questions

Hands-on

Task: Text generation

Prompt: Write an email for absence during a meeting

Large Language Model (LLM)

LLM categorization by output

- Code Generation Models
 - Often trained on large code datasets, such as GitHub
 - Generate new code or fix bugs in existing code

Large Language Model (LLM)

LLM categorization by architecture

- Decoder only model

- Content creator
- Good at writing content
- Not good at understanding and learning

e.g., GPT family models, such as GPT-3

Large Language Model (LLM)

LLM categorization by architecture

- Encoder only model

- Understand and learn content
- Not good at generating

e.g., BERT

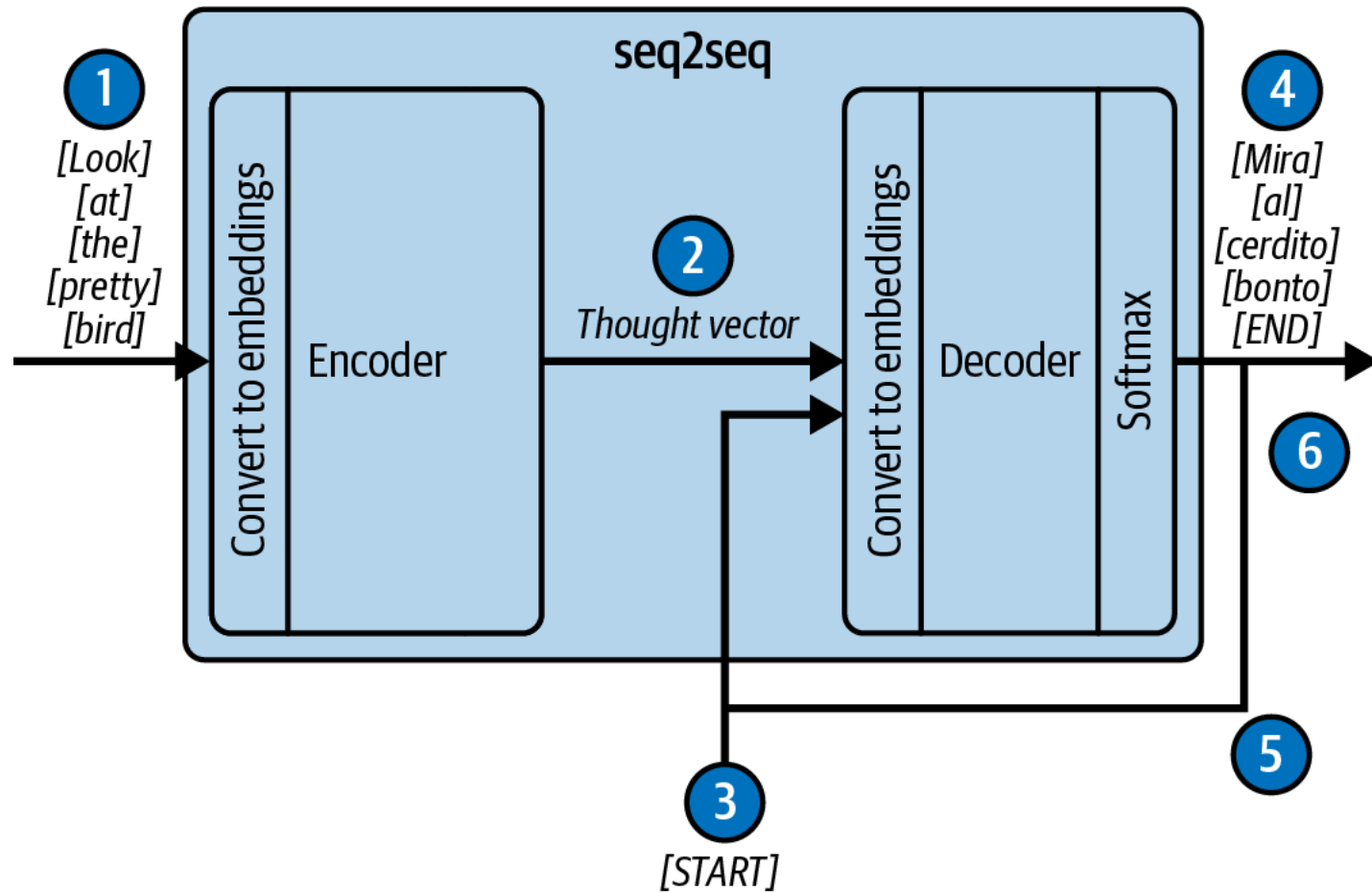
Large Language Model (LLM)

LLM categorization by architecture

- Encoder-Decoder model

- Understand and learn content
- Generate content

e.g., BART, T5



Large Language Model (LLM)

Explore LLMs via Hugging Face

<https://huggingface.co/>

- Compare different models
- Evaluate
- Improve performance
- Deploy

Large Language Model (LLM)

LLM Approaches:

- Prompt engineering
- Retrieval Augmented Generation (RAG)
- Fine-tuned LLM
- Trained LLM

Large Language Model (LLM)

Prompt Engineering:

Pre-trained LLMs

- Provide **context** to prompts to get desired LLM responses
 - Most cost-effective approach
- **Context**: Prompt with a **detailed request and examples** →
LLM provides more accurate and
Closer responses to expectations

Large Language Model (LLM)

Prompt Engineering:

Zero-shot Learning:

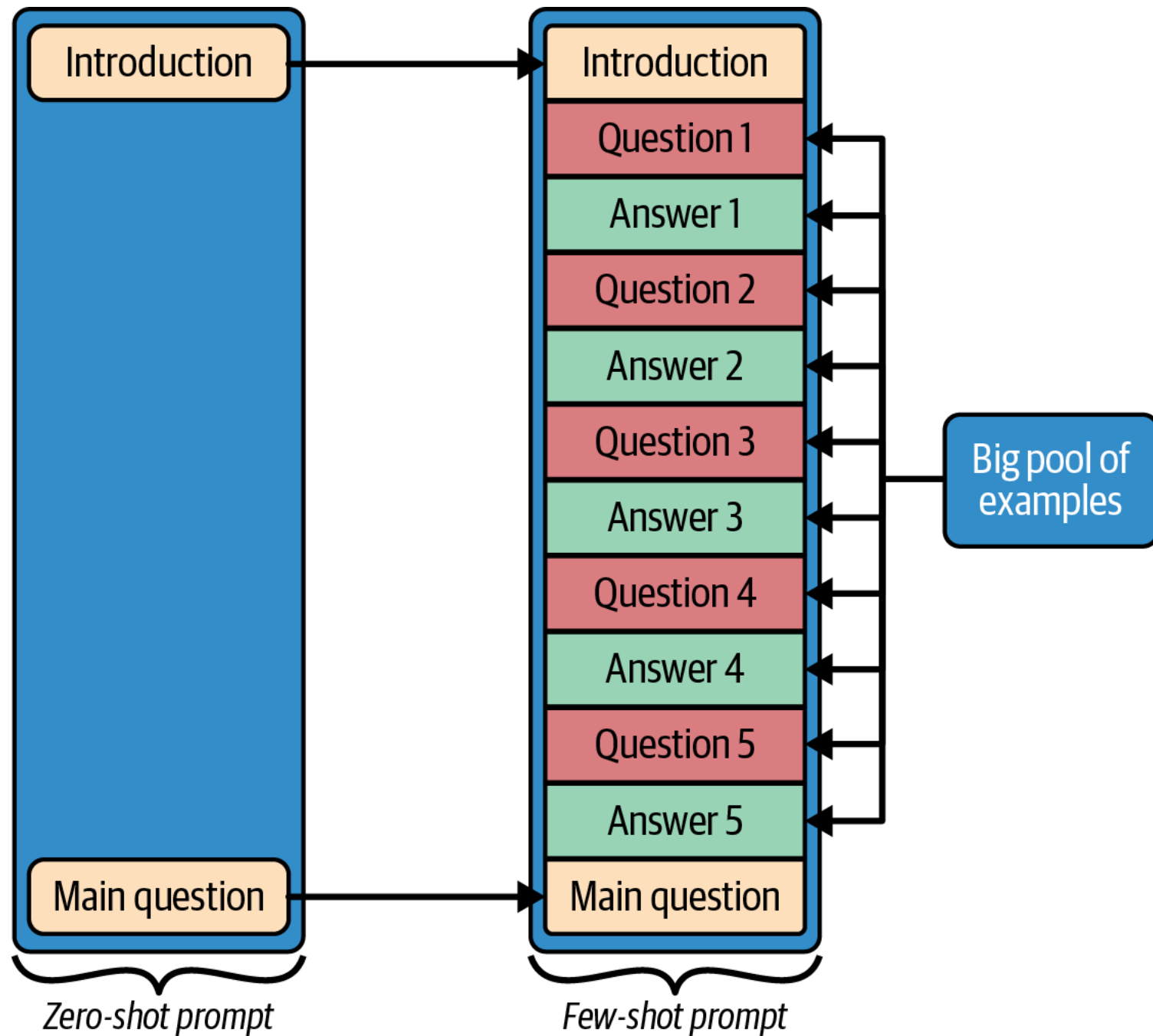
- Calling LLM with a **prompt**

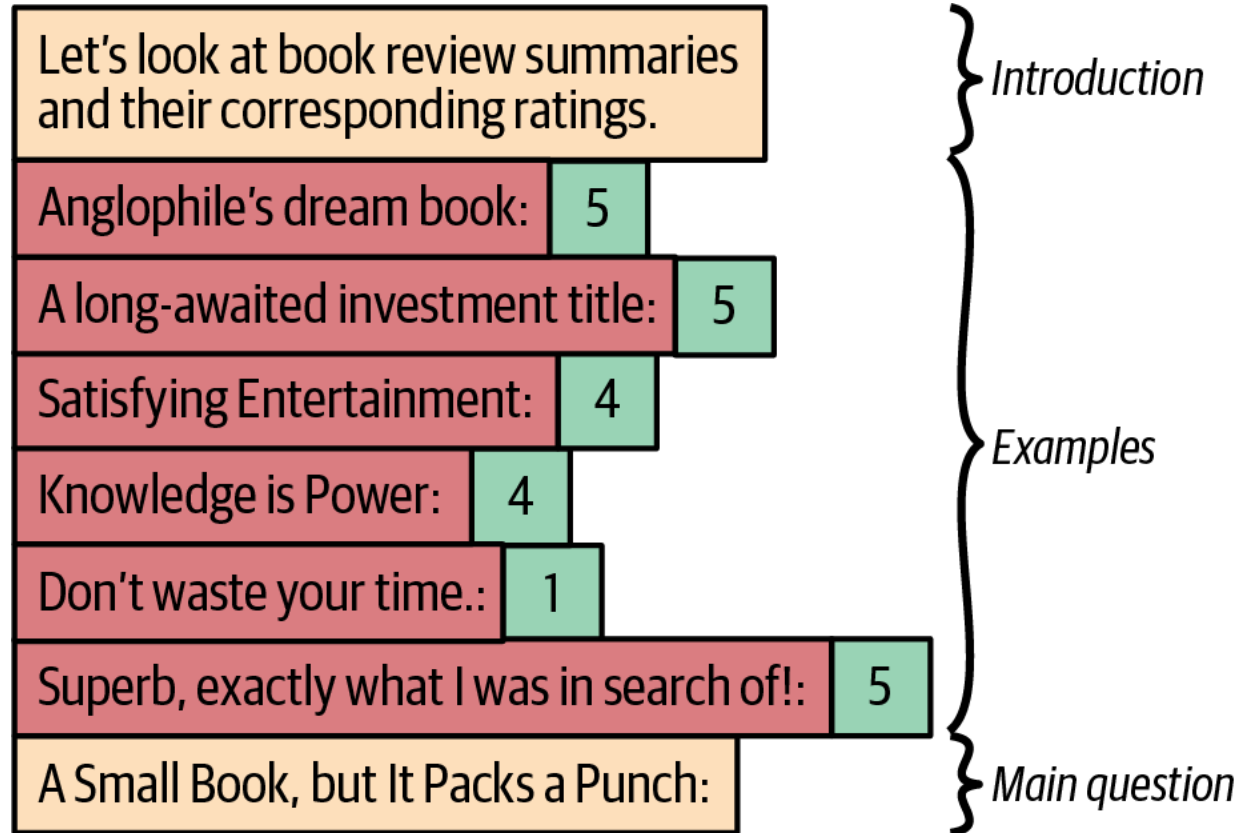
One-shot Learning:

- Prompt includes **one example**

Few-shot Learning:

- Prompt includes **multiple examples**





Large Language Model (LLM)

Retrieval Augmented Generation (RAG):

- Retrieve **relevant data** and add it to prompts to get desired LLM responses

Technique: augments the prompt with **external data**

- Chunks of documents, based on **prompt length limits**

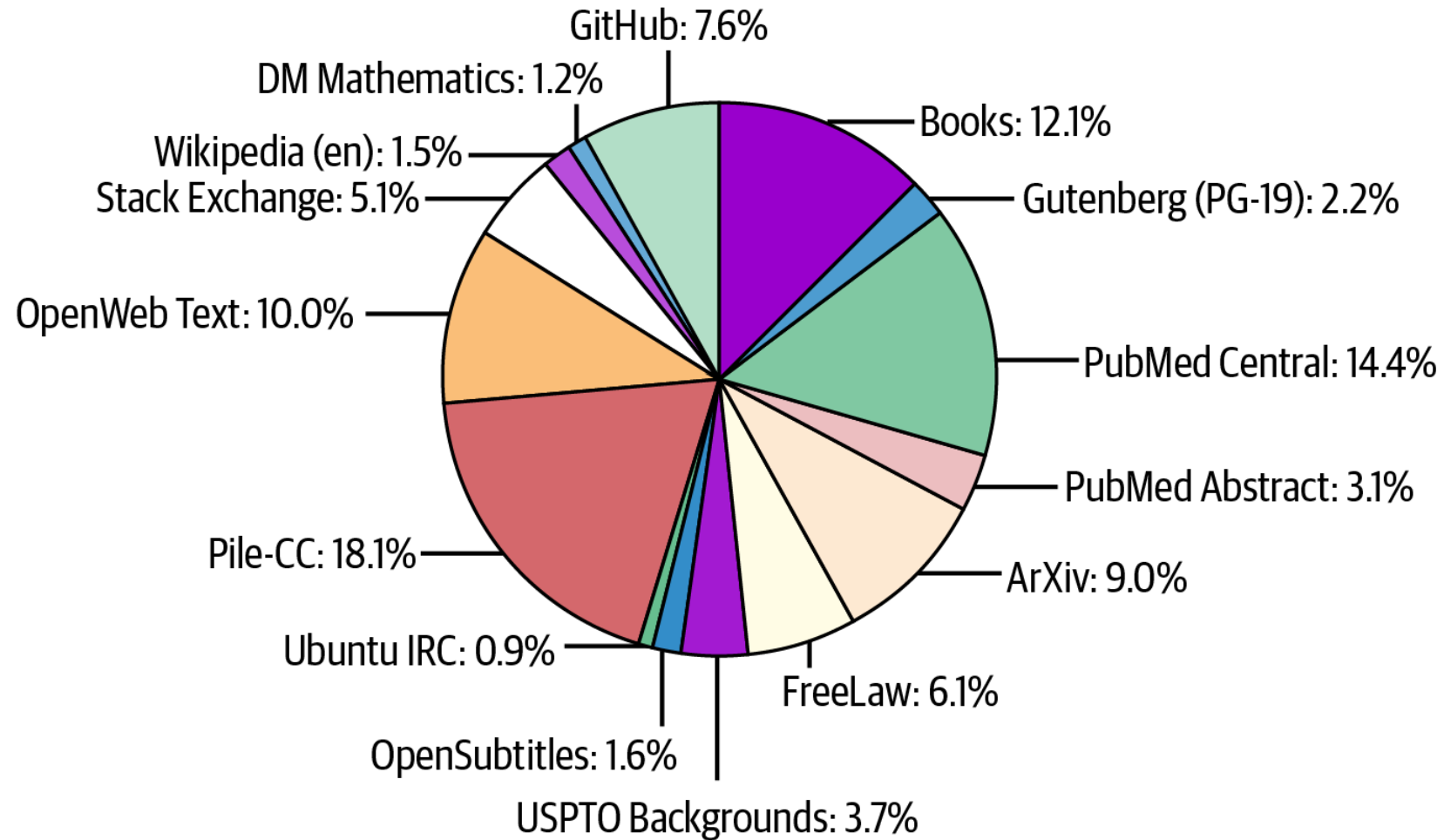
Large Language Model (LLM)

Retrieval Augmented Generation (RAG):

Why RAG?

- LLMs were not trained on non-public information
 - Like enterprise data: No access
- Solution: RAG

Effective size of sources in "The Pile"

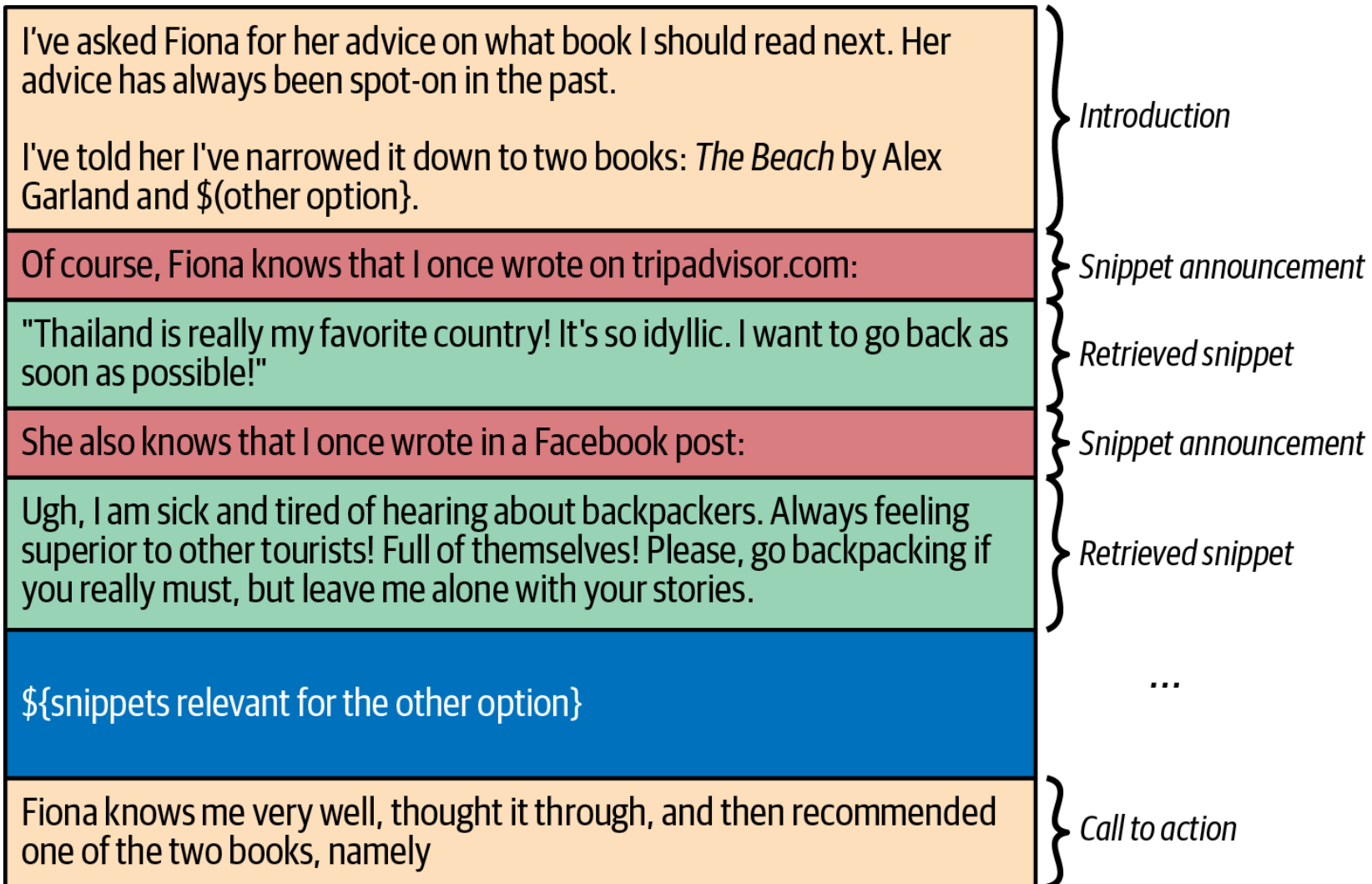


Large Language Model (LLM)

Retrieval Augmented Generation (RAG):

How?

- Vector Database Tools
- Retrieve **useful chunks** from pre-defined data sources and add them to the **prompt context**



Large Language Model (LLM)

Fine-tuned LLM:

- **Train** LLM further on relevant data to improve the model to be more exact and responsive
 - Likely to be **costly**

How?

- Transfer Learning

Large Language Model (LLM)

Fine-tuned LLM:

Transfer Learning

- Employed to **adapt** the model to a downstream task or to a specific task
 - Produces a **new model**, with **updated** weights and biases
- Training examples: getting labeled data **increases cost**

Large Language Model (LLM)

Fine-tuned LLM:

Decision trade-offs?

- Cost-effective solution (less capable) over high-performance
- Latency: long prompts are not desirable
- Prompt length limit constraint to accommodate more examples
- Updated Model:
 - Have lots of high-quality labeled data and computing resources

Large Language Model (LLM)

Trained LLM:

- Training an LLM from scratch
 - Most difficult, most complex approach
 - Massive data, skilled resources, and computational power
- Decision trade-offs?
 - Domain-specific use case
 - Large amount of domain-centric data

Large Language Model (LLM)

Hallucinations (Fabrications)

- A term used to describe when an LLM produces **absurd content** or **factually wrong** information
- Verifiable based on **reliable sources** of information

Let's try historical questions?

Hands-on

Task: Hallucinations?

Prompt: Any examples where ChatGPT hallucinated?

Hands-on

Task: Creative Ideation

Prompt: What was the winning strategy in the IPL final match of 2026

Hands-on

Tokenizer Playgrounds

<https://huggingface.co/spaces/Xenova/the-tokenizer-playground>

<https://platform.openai.com/tokenizer>

<https://claude-tokenizer.vercel.app/>

Hands-on

AI Playgrounds

<https://platform.openai.com/home>

https://aistudio.google.com/prompts/new_chat

https://huggingface.co/models?pipeline_tag=text-generation&sort=trending

Prompt Engineering

Summary:

LLMs are prompt-based:

- Send a text input (prompt)
- Get back the AI response (completion)

Chat iteratively: multi-turn conversations

- Refine the prompt to match the response with expectations

PROMPT

user request received
as a text input

LANGUAGE MODEL

8,192 Tokens Context
Pre-training data cut-off: End 2023

RESPONSE

generated content returned
as user response



Meta
Llama 3-8B



Prompt Engineering

Summary:

Prompts:

- Programming interface of Generative AI services

Prompt Engineering:

- Designing and refining prompts to guide Generative AI models toward generating desired outputs

Prompt Engineering

Summary:

Tokenization: LLM's tokenizer breaks the prompt into **tokens**

Encoding: Each token is mapped to a unique numerical integer **ID** based on the LLM's **pre-defined vocabulary**

Processing: LLM **processes these numbers** to generate its response

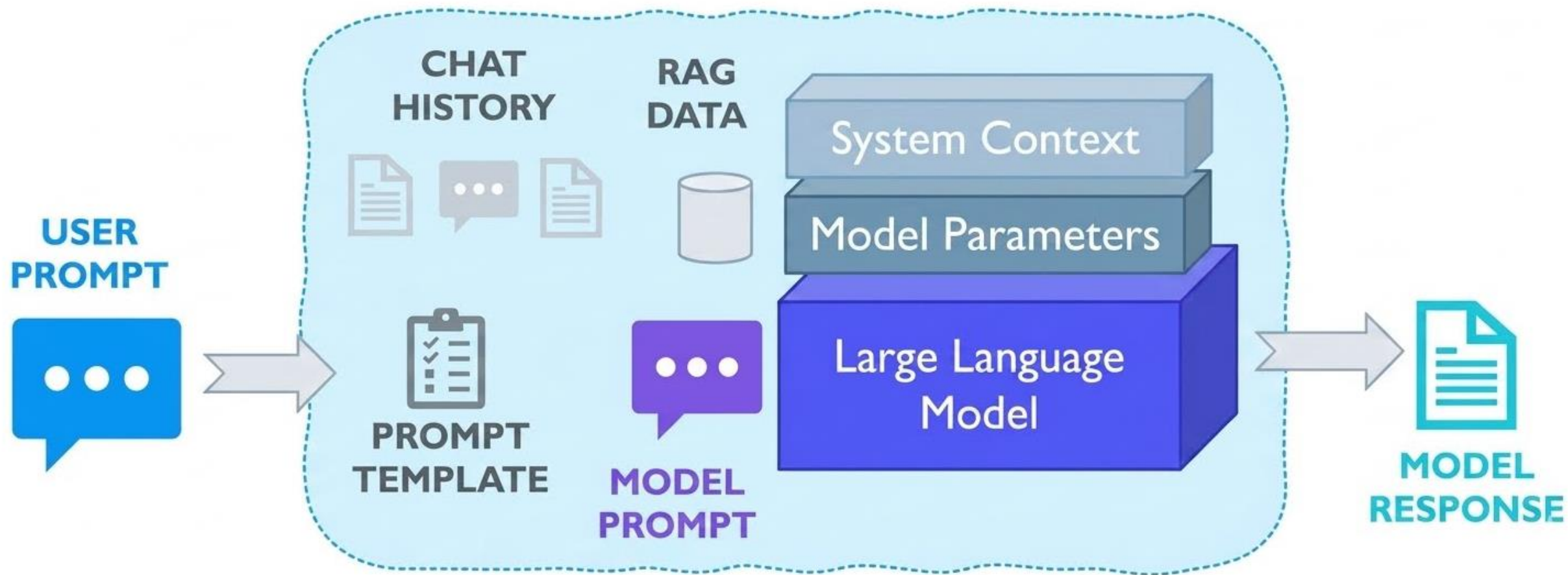
Prompt Engineering

Summary:

Embedding: Numerical IDs serve as a lookup index to retrieve a token's vector embedding

LLM Vocabulary: Locked during the initial training phase of LLM, and never changes during inference

PROMPT ENGINEERING VARIABLES



Prompt Engineering

Instruction-Tuned LLMs:

- LLMs that have been **fine-tuned** with specific instructions to improve their *response accuracy* and *relevance*
- Reinforcement Learning with **Human** Feedback (RLHF)
 - Trains LLM to **follow** instructions and **learn** from feedback
 - Fine-tunes using input/output pairs

Prompt Engineering

Instruction-Tuned LLMs:

- Reinforcement Learning with Human Feedback (RLHF)
- Suitable for Practical Applications: responses become more relevant to user objectives

Prompt Engineering

Context Window

- Maximum amount of information measured in tokens

That LLM can hold in its memory

During a single interaction or session

- LLM's active workspace or short-term memory

Prompt Engineering

Context Window

- All Prompt components must fit within this limit to be considered by LLM
 - Instructions, system persona, uploaded files, conversation history, external data (RAG)
- Exceed the context window

LLM starts to forget to make room for new input

- Earliest parts of the prompt or conversation history

Prompt Construction

Basic Prompt

- An input text sent to the model with no other context

Complex Prompt

- A collection of messages:
 - Input/output pairs: indicate user input and LLM response
 - System message: sets the context for LLM behavior/persona

Hands-on

Prompt: Jana Gana Mana

Prompt: Dhurandhar

Prompt Construction

Complex Prompt

- System context may have an **impact on the quality** of completions, just like user inputs

Instruction Prompt

- Use input text to **specify more details** on a task, providing better guidance to the LLM
 - Instead of a simple text query requesting information

Hands-on

Prompt: Summarize cloud computing

Prompt: Summarize cloud computing. Provide 3 bullet points with different types of cloud computing

Prompt: Summarize cloud computing. Provide 3 bullet points with different types of cloud computing. Return the output as a JSON file

Prompt Construction

Open-ended Prompt Design

- Relies on the LLM's decision on

Which part of its pre-trained dataset is relevant

Primary Content Design

- Input text is divided into two parts:
 - Instructional Part: indicates action
 - Relevant Content Part: influences action

Hands-on

Prompt: Explain Blockchain technology

Prompt:

Add Response of last Prompt

Summarize this in two short sentences

Prompt Construction

Primary Content Design with Examples

- Feed LLM with **examples** of desired output, and **let it infer** patterns
 - Mostly, no **explicit** instruction
- Prompt consists of:
 - A few **examples** of the desired output
 - Start of a **new example**: an **implicit task** description

Prompt Construction

Primary Content Design with Examples

- **Type of Learning** based on the number of examples
 - Zero-shot Prompting (explicit instruction may be needed)
 - One-shot Prompting
 - Few-shot Prompting

Hands-on

Prompt: "Satyam Shivam Sundaram". Translate to English

Prompt:

The batsman missed the ball: Cricket

The player couldn't pick up a drop: Badminton

The goalie defended using his stick:

Prompt Construction

Primary Content Design with Cues

- Provide **cues** that reflect the desired response format
 - Instead of examples

Primary Content Design with Templates

- Prompt Template: a **pre-defined** recipe for a prompt
 - Store and **reuse** as needed
 - Drives more **consistent** user experiences **at scale**

Hands-on

Prompt: Write a Vikram Betal story

Prompt:

Add Response of last Prompt

The main lesson of the story is

Prompt Construction

Primary Content Design with Templates

Simplest Form

- A **collection** of **prompt examples** consisting of
 - **Interactive** prompt components: user and system messages
 - API-driven **request format**: supports reuse

Prompt Construction

Primary Content Design with Templates

Complex Form

- Contains placeholders, replaceable with data from varied sources to generate a prompt dynamically
 - User input, system context, or external data sources
- Create a library of reusable prompts
 - Drive consistent user experiences programmatically at scale

Prompt Construction

Primary Content Design with Templates

Next Step: Real value of templates

- Create & publish **prompt libraries** for vertical application **domains**
- Optimized **prompt template** reflecting application-specific context
- **Examples** that make the responses more relevant and accurate

Prompt Construction

Secondary Content Design

- Input text is divided into three parts:
 - Instructional Part: indicates action
 - Primary Content Part: influences action
 - Secondary Content Part: influences action in some way

Prompt Construction

Secondary Content Part can be

- Tuning parameters, formatting instructions, topic taxonomies
- Help the model tailor its response to suit the desired user objectives or expectations

Prompt Construction

Chain-of-thought Prompting

- Guides LLM on how to break down a problem into steps

Prompt: Ramesh has 5 apples, throws 3 apples, gives 2 to Binod, and Binod gives one back. How many apples does Ramesh have?

Prompt Construction

Chain-of-thought Prompting

Apply chain-of-thought:

- Provide a similar example
- Show the calculation steps
- Add the original prompt

Prompt Construction

Prompt:

Mohan has 7 apples, throws 1 apple, gives 4 apples to Soham, and Soham gives one back: $7 - 1 = 6$, $6 - 4 = 2$, $2 + 1 = 3$

Ramesh has 5 apples, throws 3 apples, gives 2 to Binod, and Binod gives one back. How many apples does Ramesh have?

Prompt Construction

Generated Knowledge Prompting

- Guide LLM by giving generated facts or knowledge to improve the response
- **Templates** containing variables to generate facts or knowledge
- Employ **APIs** to fetch actual values
- **Optimize** the prompt by clarifying what it can allow
- **Trial and error**

Prompt Construction

Prompt:

{{company}}: {{company_name}}

{{products}}:

{{products_list}}

Please suggest an insurance given the following budget and requirements:

Budget: {{budget}}

Requirements: {{requirements}}

Prompt Construction

Insurance company: XYZ Insurance

Insurance products (cost per month):

- Car, cheap, 500 USD
- Car, expensive, 1100 USD
- Home, cheap, 600 USD
- Home, expensive, 1200 USD
- Life, cheap, 100 USD

Please suggest an insurance given the following budget and requirements:

Budget: \$1000

Requirements: Car, Home, and Life insurance

Prompt Construction

Insurance company: XYZ Insurance

Insurance products (cost per month):

- type: Car, cheap, cost: 500 USD
- type: Car, expensive, cost: 1100 USD
- type: Home, cheap, cost: 600 USD
- type: Home, expensive, cost: 1200 USD
- type: Life, cheap, cost: 100 USD

Please suggest an insurance given the following budget and requirements:

Budget: \$1000 restrict choice to types: Car, Home

Prompt Construction

Least-to-most Prompting

- Initial Prompt: ask LLM to **break down** a bigger problem into subproblems
- Prompt it to perform these steps in order

Prompt: How to apply machine learning in 6 steps?

Prompt Construction

Self-refine Prompting

- Initial prompt: ask LLM to solve a problem
- LLM response
- Critique the LLM response and ask it to improve
- LLM considers the critique and comes up with solutions
- Repeat as needed for refinement

Prompt Construction

Prompt: Create a Web API for products and customers

Prompt: Suggest improvements to the above code

Prompt Construction

Maieutic Prompting

- Prompt LLM to answer a question
- For each part of the answer, prompt it to **explain in depth**
- Check for **inconsistencies**, discard those parts
- **Repeat** for all the parts, or until a satisfactory response

Prompt Construction

Prompt: How can I create a study plan to prepare for a competitive exam in 5 steps?

Prompt: Explain the second step in more detail. What are the realistic goals in detail?

Prompt: During a competitive exam, what is the main goal and why?

Prompt: What are the two biggest goals in a competitive exam?

Prompt Construction

Tree of Thoughts Prompting

- Elevates reasoning
 - Moving beyond a single, **linear path** of thought
 - Exploring multiple **branching** possibilities
- Treats the problem-solving process like a **decision tree**

Prompt Construction

Tree of Thoughts Prompting

- Ask LLM to
 - Explore various potential strategies
 - Evaluate them
 - Decide which paths to continue and which to abandon

Prompt Construction

Prompt: I need to plan a healthy dinner. Suggest 3 different 20-minute recipes, evaluate the pros and cons of each, and tell me which one is best for someone on a budget

Prompt Construction

RT-C-F (Role, Task, Context, Format) Framework

- Design precise, reliable instructions for prompts

Role: LLM behavior/persona

- To set the tone, vocabulary, and depth of analysis

Task: specific, actionable objective

- Defines the core output and prevents scope creep

Prompt Construction

RT-C-F (Role, Task, Context, Format)

Context: background, constraints, and why part

- Provides guardrails and the necessary information

Format: How the output should be structured

- Immediately usable response, e.g., table, code, email

Prompt Construction

Prompt:

Role: Act as a professional executive assistant.

Task: Write an **email** to a client explaining that our project delivery will be delayed by one week.

Context: The delay is due to unexpected server maintenance. We have already completed the design phase, and this extra time ensures the final security testing is robust.

Format: Keep it under 150 words, use a polite and professional tone, and include a clear subject line

Hands-on

Let's shift from No-Code to Code-First

Open Google Colab

Thanks